

Todo

- Was ist eine LTS Semantic?

Paper Ablauf

1. Was ist RPC
2. Warum sind Deadlocks blöd
3. Warum sind die schwer zu finden
4. *false positives* in sound static analysis

Aufbau

1. Motivation
2. Formalisierung erarbeiten
3. Soundness & Completeness Proofs (Sketch?)
4. Deadlock Algorithm

Keywords

- Message Passing: Query, Response, Cast
- Remote Procedure Calls
- Deadlock: Cyclic wait
- Monitors: Analyse traffic
 - Probes: Communication with other monitors
- Black-Box: No introspection

Umsetzung

- Viele Visualisierungen vom Netzwerk wären nice
 - Aus dem Paper?
 - Easy aber arsch Quali
 - Mit diesem Graphen CLI-Tool?
 - Mit Anki?
 - Mit Typst?

Quirks

- The monitored path might be different!

Proofs and Definitions

- Criterion 2.1 (Monitor Instrumentation Transparency)
 - Operational Correspondance vs. Simulation Relation
 - soundness allows some extra steps
- Criterion 2.2 (Preciseness of Deadlock Detection)
 - completeness allows some extra steps
- Lock
- Deadlock
- Deadlockset
- Lemma 3.8 (Persistence of deadlocks).

Slides

Motivation

- Deadlocks sind blöd
- Bild von Deadlock

Setting

- Netzwerk
- (S)RPC- Bild von Deadlock
- Blackbox

Monitors

1. Distributed without introducing centralized component
2. Blackbox and outline: Detect deadlocks by just observing the incoming and outgoing messages of the service they monitor
3. Transparent: Don't alter the execution
4. Precise: Detect Deadlock iff it existst

Formalism

Network: N Monitored Network: $\hat{N}$ Transitions: $N \xrightarrow{\alpha} N'$ Many
Transitions(path): $N \xrightarrow{\sigma} N'$ Instrumentation: $\mathcal{M}$ De-instrumentation:
 $\mathcal{M}^{-1}$

Nodes

Formalism: Services

$$\begin{aligned} t &:= Q \mid R \mid c \mid n \\ n &:= n_0 \mid n_1 \mid \dots \mid n^{\perp} \\ n^{\perp} &:= n \mid \perp \\ P &:= \dots \mid q := \varepsilon \mid n(t) \mid q ++ q \\ S &:= \langle q^i \mid P \mid q^o \rangle \end{aligned}$$

Formalism: LTS Semantic of abstract SRPC Process G

$G := \text{Ready} \mid \text{Working} \mid \text{Locked}$ $\gamma := ?n(t) \mid !n(t) \mid \tau$ We turn a Service S into a Process G

Formalism: LTS Semantic of Service

$$\delta := ![n(t)] \quad \beta := ?n(t) \mid !n(t) \mid \tau(?n(t)) \mid \tau(!n(t)) \mid \tau(\tau)$$

Networks

Formalism: Network

- We write $N(n)$ to denote the service named n within N
- We write N for the set of all networks.

$$\alpha := \tau(\gamma @ n) \mid n_0 \xrightarrow{(t)} n_1$$

Formalism: Deadlocks

- TODO

Monitors

Formalism: Monitors

$\text{hat}(m) := ?n(t) \mid !n(t) \mid ?n(\text{hat}(p)) \mid !n(\text{hat}(p))$
 $\text{hat}(q) := \varepsilon \mid \text{hat}(m) \mid \text{hat}(q) +$
 $+ \text{hat}(q) \text{ hat}(S) := \langle \text{hat}(q) \mid P \mid S \rangle$

Monitored service action: $\text{hat}(\beta) := \beta \mid ?n(\text{hat}(p)) \mid !n(\text{hat}(p)) \mid \text{hat}(\tau)(?$
 $n(t)) \mid \text{hat}(\tau)(!n(t)) \mid \text{hat}(\tau)(?n(\text{hat}(p)))$ τ being the internal actions

Monitor algorithm function: $\text{hat}(\mathcal{A})$

- This function is total

Formalis: Monitored Network action

$\hat{\alpha} := \alpha \mid \hat{\tau}(\gamma @ n) \mid n_0 -(\hat{p})-> n_1$ monitored path: $\hat{\sigma}$

Instrumentation

Monitor instrumentation: $\mathcal{M} : \mathcal{N} \rightarrow \text{hat}(\mathcal{N})$ Deinstrumentation: $\mathcal{M}^{(-1)}$

Monitor stripping operation: $\text{hat}(S)\downarrow: \langle \hat{q} \mid \hat{M} \mid \langle q^i \mid P \mid q^o \rangle \rangle = \langle q^i +$
 $+ \text{filter}^{i(\hat{q})} \mid P \mid q^o + + \text{filter}^{o(\hat{q})} + + q^o \rangle$ Stripping of monitor path:
 $\text{hat}(\sigma)\downarrow: \dots$

Theorem 4.8 (Black-box instrumentation transparency — completeness).

The Algorithm

- Monitor states $\text{hat}(M)$: probe | waiting | alarm
- Implementation of $\text{hat}(\mathcal{A})$

Wellformedness of SRPC

- Definition: Initial Network
- Definition: Client
- Definition: SRPC well-formedness
- Definition: Wellformed SRPC Network
- Theorem: Wellformedness as an *invariant*
- Definition: Complete Lock Knowledge
- Definition: Alarm Condition
- Definition: Sound lock knowledge
- Theorem: Sound lock knowledge is an *invariant*
- Theorem 6.13 (Deadlock detection preciseness)

- Theorem 6.13 (eventual deadlock reporting)

Evaluation

- Overhead is neglectible